

Aula 5 | Laços de Repetição



Objetivos da aula de hoje

Ao final, você deve conseguir:

- Resolver o problema: criar um jogo da adivinhação que mantém o usuário tentando até acertar – com feedback “muito alto” ou “muito baixo”. Entender e aplicar: WHILE (verifica antes) e DO-WHILE (executa ao menos uma vez), variável de controle e condição de parada. Adicionar validações na agenda que repetem a entrada até que os dados sejam válidos.

Problema gerador

Jogo da adivinhação e agenda com entrada validada

Pergunta guia

Como fazer um programa que sorteia um número entre dois inteiros e mantém o usuário respondendo até que ele acerte – dando feedback se o palpite foi maior ou menor que o número sorteado?

- Estudo de caso 1: Jogo da adivinhação com while e feedback de maior/menor
- Estudo de caso 2: Agenda que pede o dado novamente até que seja válido
- Expositiva: While / Do-while (semelhanças e diferenças)
- Prático: Exercícios de fixação (while/do-while)

Conceitos

Laços de Repetição

- WHILE – repete enquanto a condição for verdadeira (verifica antes)
- DO-WHILE – executa ao menos uma vez, verifica a condição depois
- Variável de controle (contador, flag, acumulador)
- Condição de parada – evita loop infinito
- break (sair do laço) e continue (pular iteração atual)
- Math.random() e Math.floor() – sortear inteiros em JS
- while(true) com break – padrão válido para laços abertos

Laço WHILE em JavaScript

WHILE:

O while verifica a condição ANTES de executar o bloco. Se já for falsa de início, o bloco nunca executa. Sempre atualize a variável de controle dentro do laço – sem isso o programa trava em loop infinito.

```
let i = 1;
while (i ≤ 5) {

    console.log('Número: ' + i);
    i++; // ← SEM isso: loop infinito!
}
// Saída: Número: 1, 2, 3, 4, 5
```

```
// O while pode nunca executar:
let x = 10;
while (x < 5) { // false de início
    console.log(x); // nunca executa
}
```

```
// while(true) com break – padrão válido:
while (true) {
```



Laço DO-WHILE em JavaScript

DO-WHILE vs WHILE: use do-while quando a primeira execução deve acontecer sempre – como pedir dados ao usuário via prompt(). Use while quando o bloco pode não precisar rodar (ex.: lista vazia ou array já processado).

O do-while executa o bloco, DEPOIS verifica a condição.

Garantia: o bloco roda pelo menos uma vez, sempre.

```
let numero;
do {
  numero = Number(prompt('Digite um positivo:'));
  if (numero ≤ 0) {
    console.log('⚠ Precisa ser > 0!');
  }
} while (numero ≤ 0);
console.log('Válido: ' + numero);
```

```
// Comparação rápida:
// while (cond) { }      → testa ANTES
// do { } while (cond);  → testa DEPOIS
```

```
// Use do-while para: prompts ao usuário,
```

```
// menus interativos, validações de entrada.
// O bloco SEMPRE executa ao menos 1 vez.
```



Estudo de Caso 1: Jogo da Adivinhação

`Math.random()` retorna decimal entre 0 (incluso) e 1 (excluso). `Math.floor()` arredonda para baixo. `Math.floor(Math.random() * 10) + 1` gera inteiros de 1 a 10. A flag `acertou` controla a saída do `while` – outra opção seria usar `break` diretamente.

```
const MIN = 1, MAX = 10;
const secreto = Math.floor(Math.random() * MAX) + MIN;
let tentativas = 0;
let acertou = false;

while (!acertou) {

  const palpite = Number(prompt(`Adivinhe (${MIN}-${MAX}):`));
  tentativas++;
  if (palpite === secreto) {
    console.log(`\u2713; Acertou em ${tentativas} tentativa(s)!`);
    acertou = true;
  } else if (palpite < secreto) {
    console.log('\u2192; Muito baixo! Tente de novo.');
  } else {
    console.log('\u2192; Muito alto! Tente de novo.');
  }
}
```



Estudo de Caso 2: Agenda com Entrada Validada

DO-WHILE é ideal para entrada do usuário: o prompt deve ser exibido ao menos uma vez antes de validar. A condição no while decide se repete. Use trim() para remover espaços acidentais.

```
const agenda = [];  
  
// 1) Pedir nome até ser válido  
let nome;  
do {  
  nome = prompt('Nome do contato:');  
  if (!nome || nome.trim() === '') {  
    console.log('⚠ Nome não pode ser vazio!');  
  }  
} while (!nome || nome.trim() === '');  
// 2) Pedir telefone até ter ao menos 8 dígitos  
  
let tel;  
do {  
  tel = prompt('Telefone (mín. 8 dígitos):');  
} while (!tel || tel.trim().length < 8);  
  
agenda.push({  
  nome: nome.trim(),  
  
  tel: tel.trim(),  
});  
console.log('Contato adicionado:', agenda[0]);
```



WHILE vs DO-WHILE — Quando usar cada um?

Escolha o laço certo para cada situação:

- WHILE — condição pode ser falsa de início (ex.: lista vazia, arquivo já processado)
- DO-WHILE — bloco deve rodar ao menos 1x (ex.: prompt ao usuário, menu)
- Ambos suportam break (sair do laço) e continue (pular iteração)

Exemplos comparativos:

```
// WHILE — pode nunca executar:
```

```
let i = 10;
```

```
while (i < 5) { console.log(i); } // nunca executa
```

```
// DO-WHILE — executa sempre ao menos 1x:
```

```
let j = 10;
```

```
do {
```

```
    console.log(j); // imprime 10
```

```
} while (j < 5); // false → para após 1 iteração
```

```
// while(true) + break = loop até acertar:
```



Exercícios de fixação

Grupo 1: “Tabuada interativa”

Dado `base` e `limite`, imprima a tabuada usando while.

EX: `const base = 7;    const limite = 10;`

1. Use `while` para imprimir `base × i = resultado` de $i=1$ até `limite`.
2. Antes do laço, valide se `base` e `limite` são números positivos (use `if`).
3. Adicione um contador de linhas impressas e mostre o total ao final.
4. Use `break` se o resultado ultrapassar 100 – pare e avise o usuário.
5. Reescreva usando `do-while`: qual é a diferença prática neste caso?
6. Quando o `while` nunca executaria aqui? Escreva um exemplo que demonstre isso.
7. ****Desafio:**** peça `base` e `limite` via `prompt` com `do-while`, validando que são inteiros entre 1 e 20.

Feito é melhor que perfeito: entregue uma versão simples, depois melhore!

Exercícios de fixação

Grupo 2: “Validação de senha”

Implemente um sistema de verificação de senha com limite de tentativas.

ex:

```
const SENHA_CORRETA = "js2025";  
let tentativas = 0;  
const MAX_T = 3;  
let acesso = false;  
// use while aqui...  
// ...
```

1. Use ``while`` com condição dupla: ``tentativas < MAX_T && !acesso``.
2. A cada tentativa errada, avise quantas restam: ``MAX_T - tentativas``.
3. Use ``break`` para sair imediatamente quando a senha estiver correta.
4. Ao final: se ``acesso`` for true → “Liberado!”; senão → “Conta bloqueada.”
5. Reescreva com ``do-while``. A lógica muda? Em quê?
6. Use ``&&`` na condição do while – por que ambas as partes são necessárias?
7. ****Desafio:**** registre ``Date.now()`` antes do laço e calcule o tempo total em segundos.

Feito é melhor que perfeito: entregue uma versão simples, depois melhore!

Exercícios de fixação

Grupo 3: “Jogo de adivinhação avançado”

Expanda o jogo do Estudo de Caso 1 com limite de tentativas e validação de entrada.

ex:

```
const MIN = 1, MAX = 100;  
const secreto = Math.floor(Math.random() * MAX) + MIN;  
let tentativas = 0;  
const MAX_T = 7;  
let acertou = false;  
// use while + do-while interno aqui...
```

1. Sorteie o número com ``Math.floor(Math.random() * MAX) + MIN``.
2. Use ``do-while`` interno para validar: palpite deve ser um número entre MIN e MAX.
3. Limite a 7 tentativas: ``while (tentativas < MAX_T && !acertou)``.
4. A cada rodada: imprima “Muito baixo!”, “Muito alto!” ou “Acertou!”.
5. Se acabar as tentativas sem acertar, revele o número secreto.
6. Mostre o número total de tentativas usadas ao final.
7. ****Desafio:**** classifique o resultado: 1–3 → “Gênio!”, 4–5 → “Ótimo!”, 6–7 → “Quase lá!”

Feito é melhor que perfeito: entregue uma versão simples, depois melhore!



Correção

```
const base = 7;

const limite = 10;
if (base ≤ 0 || limite ≤ 0) {
  console.log('base e limite devem ser positivos');
} else {
  let i = 1, linhas = 0;
  while (i ≤ limite) {
    const resultado = base * i;
    if (resultado > 100) {
      console.log('Resultado > 100 – parando!');

      break;
    }
    console.log(`${base} × ${i} = ${resultado}`);
    linhas++;
    i++;
  }
  console.log(`Total de linhas: ${linhas}`);
}
// Com do-while: executa ao menos 1x mesmo se i = limite + 1
// let i = 1;
// do {
//   console.log(`${base} × ${i} = ${base * i}`);
//   i++;
// } while (i ≤ limite);
// Aqui while e do-while geram o mesmo resultado:

// base e limite já foram validados antes do laço.
```



Correção

```
const SENHA_CORRETA = "js2025";
let tentativas = 0;
const MAX_T = 3;
let acesso = false;

while (tentativas < MAX_T && !acesso) {

  const entrada = prompt('Digite a senha:');
  tentativas++;
  if (entrada === SENHA_CORRETA) {
    acesso = true;

    break; // sai do while imediatamente

  } else {
    const restam = MAX_T - tentativas;
    if (restam > 0) {
      console.log(`Senha errada. Restam: ${restam}x.`);
    }
  }
}
if (acesso) {

  console.log('Acesso liberado!');
} else {

  console.log('Conta bloqueada.');
}
```



Correção

```
const MIN = 1, MAX = 100;
const secreto = Math.floor(Math.random() * MAX) + MIN;
let tentativas = 0;
let tentativas = 0, MAX_T = 7, acertou = false;

while (tentativas < MAX_T && !acertou) {

  let palpite;
  do {

    palpite = Number(prompt(`Adivinhe (${MIN}-${MAX}):`));
  } while (isNaN(palpite) || palpite < MIN || palpite > MAX);
  tentativas++;

  if (palpite === secreto) {
    acertou = true;
  } else if (palpite < secreto) { console.log('Muito baixo!'); }
  else { console.log('Muito alto!'); }

}

const cl = tentativas ≤ 3 ? 'Gênio!' : tentativas ≤ 5 ? 'Ótimo!' : 'Quase lá!';
```



Pesquisa Individual

Laços de Repetição e Controle de Fluxo

Leituras recomendadas:

- MDN while:
developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Statements/while
- MDN do-while:
[developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Statements/do ... while](https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Statements/do...while)

Perguntas de pesquisa (10–15 min)

- 1) Qual a diferença entre ``break`` e ``continue`` dentro de um laço?
(Dica: `break` sai do laço; `continue` pula só a iteração atual)
- 2) O que acontece se você esquecer de incrementar o contador em um ``while``?
- 3) Um ``while(true)`` com ``break`` interno é válido em JS? Quando usar?
- 4) Pesquise: quando o ``for`` é mais claro que o ``while``? Dê um exemplo.
- 5) Bônus: o que é uma variável "flag" (ex.: `acertou = false`)? Por que é útil em laços?



Próxima aula

Aula 6

Laços de iteração: `for` clássico, `for ... of` e `for ... in` – percorrer arrays e objetos de forma elegante.

Para chegar bem:

- Traga dúvidas sobre `while/do-while` e os exercícios
- Complete os 3 exercícios de fixação
- Pesquise a diferença entre `break` e `continue`



Obrigado